Transformacje

Transformacje

transform

Właściwość CSS umożliwiająca:

- przekształcanie elementów bez wpływu na flow dokumentu,
- operacje w przestrzeni 2D (i opcjonalnie 3D).

Transformacje:

- są renderowane z użyciem GPU,
- nie powodują przeliczania layoutu (lepsza wydajność niż margin, top itd.)

```
.element {
  transform: translateX(50px);
}
```

Flow dokumentu to **naturalny model renderowania drzewa DOM**, w którym elementy są układane zgodnie z:

- kolejnością w kodzie HTML
- ich domyślnym typem wyświetlania (`display`)

Transformacje

Translacja (przesunięcie)

- przesuwa element względem jego pozycji początkowej
- nie wpływa na inne elementy

```
transform: translate(x, y);
```

```
.element {
  transform: translate(20px, 40px);
  transform: translateX(50px);
  transform: translateY(20px);
  transform: translateY(50%, 20%);
}
```

- procentowe wartości w `translate(X%, Y%)` odnoszą się do wymiarów samego elementu, a nie rodzica.
- Jest to bardzo często wykorzystywane do centrowania elementów absolutnych (`top: 50%; left: 50%; transform: translate(-50%, -50%);`)

Transformacje

Translacja (przesunięcie)

- przesuwa element względem jego pozycji początkowej
- nie wpływa na inne elementy

```
transform: translate(x, y);
```

```
.element {
  transform: translate(20px, 40px);
  transform: translateX(50px);
  transform: translateY(20px);
  transform: translateY(50%, 20%);

}
```

Skalowanie

- zmiana rozmiaru elementu
- wartości:
  - 1 → brak zmiany
  - <1 → zmniejszenie
  - >1 → powiększenie

```
.element {
  transform: scale(1.5);
  transform: scale(2, 0.5);
}
```

Transformacje

Rotacja

- obrót wokół punktu odniesienia (transform-origin)
- jednostki: deg, rad, turn

```
.element {
  transform: rotate(45deg);
}
```

Punkt odniesienia – `transform-origin`

- określa punkt odniesienia, wokół którego wykonywana jest transformacja
- domyślnie środek elementu: 50% 50%
- wartości:

```
transform-origin: center;
transform-origin: top left;
transform-origin: 50% 50%;
```

```
.element {
  transform-origin: left top;
  transform: rotate(45deg);
}
```

Transformacje

Pochylenie (skew)

Deformuje element poprzez przesunięcie jego osi, co daje efekt "przechylenia".

```
.element {
  transform: skew(20deg, 10deg);
}
```

```
.element {
  transform-origin: left top;
  transform: skew(20deg, 10deg);
}
```

Transformacje

Łączenie transformacji

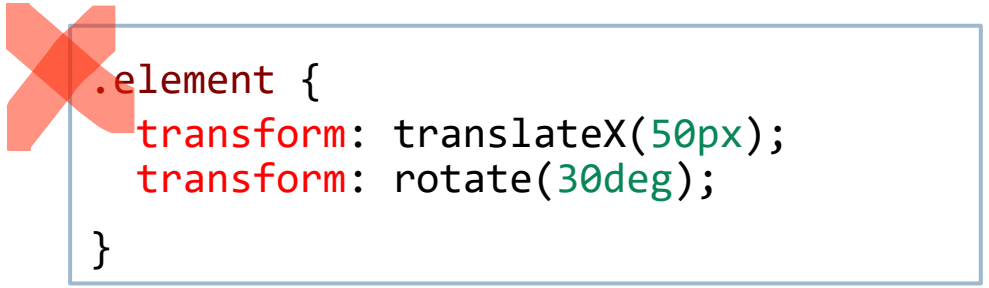
- Transformacje można łączyć w jednej deklaracji.
- Są one wykonywane sekwencyjnie od lewej do prawej.
- kolejność operacji wpływa na wynik końcowy — zmiana kolejności może dać zupełnie inny efekt wizualny.

```
.element {
transform: translateX(50px) rotate(30deg) scale(1.2);

}
```

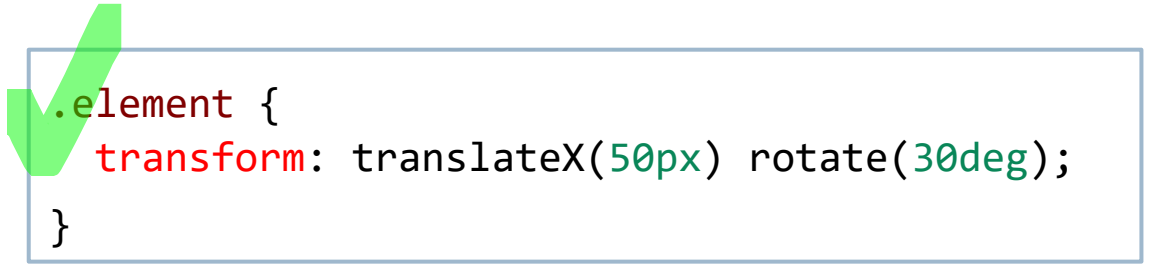
Często spotykany błąd:

- każda kolejna deklaracja nadpisuje poprzednią — należy łączyć funkcje w jednej właściwości



```
.element {
  transform: translateX(50px);
  transform: rotate(30deg);

}
```



```
.element {
  transform: translateX(50px) rotate(30deg);
}
```

Transformacje 3D

Transformacje 3D

- Pozwalają operować w przestrzeni trójwymiarowej (oś Z).
- `perspective` definiuje głębię sceny — im mniejsza wartość, tym silniejszy efekt perspektywy

```
.container {
  perspective: 500px;
}

.box {
  transform: rotateX(45deg);
}
```

Transformacje 3D

rotateY i efekt „karty”

- często używane do implementacji efektu odwracanej karty (flip card)
- aby efekty 3D (szczególnie `rotateY` i `translateZ`) były dobrze widoczne (z efektem głębi), element rodzica musi mieć ustawioną właściwość `perspective`

```
.box {
  transition: transform 0.6s;
  transform-style: preserve-3d;
}

.box:hover {
  transform: rotateY(180deg);
}
```

Transformacje 3D

translateZ i efekt wysunięcia

- przesuwa element wzdłuż osi Z — używane razem z `perspective`, daje efekt „wysunięcia” do użytkownika.

```
.box {
    transform: translateZ(50px);
}
```

Animowanie transformacji (transition)

Transition:

- Służy do kontrolowania płynności zmian właściwości CSS (np. przy najechaniu kursorem :hover).
- Pozwala na stworzenie prostej animacji w oparciu o transformacje bez użycia JavaScriptu czy skomplikowanych keyframes.

Składnia:

transition: [właściwość] [czas] [funkcja dynamiki]; (np. transition: transform 0.5s ease;)

```
.box {
  /* Płynne przejście tylko dla transformacji */
  transition: transform 0.4s ease-in-out;
}

/* Stan, do którego dążymy */
.box:hover {
  transform: scale(1.2) rotate(15deg);
}
```

Animowanie transformacji (transition)

Transition:

- Służy do kontrolowania płynności zmian właściwości CSS (np. przy najechaniu kursorem :hover).
- Pozwala na stworzenie prostej animacji w oparciu o transformacje bez użycia JavaScriptu czy skomplikowanych keyframes.

Składnia:

transition: [właściwość] [czas] [funkcja dynamiki]; (np. transition: transform 0.5s ease;)

```
.box {
  /* Płynne przejście tylko dla transformacji */
  transition: transform 0.4s ease-in-out;
}

/* Stan, do którego dążymy */
.box:hover {
  transform: scale(1.2) rotate(15deg);
}
```

Animowanie transformacji (transition)

Często spotykany błąd:

- Właściwość transition powinna znajdować się zawsze w bazowym stylowaniu elementu (np. .box), a nie w pseudoklasie (.box:hover).
- Jeśli dodamytransition tylko do :hover, element po zjechaniu kursorem powróci do stanu pierwotnego skokowo, bez animacji.

```
.box {
  /* Płynne przejście tylko dla transformacji */
  transition: transform 0.4s ease-in-out;
}

/* Stan, do którego dążymy */
.box:hover {
  transform: scale(1.2) rotate(15deg);
}
```

Ciekawostka

Ponieważ Transformacje są renderowane z użyciem GPU, dodanie **transform: translateZ(0)** może wymusić akcelerację sprzętową, co zapobiega migotaniu lub problemom z płynnością przy animacjach na słabszych urządzeniach.